1 Lesson 04-11-26

1.1 Synchronization

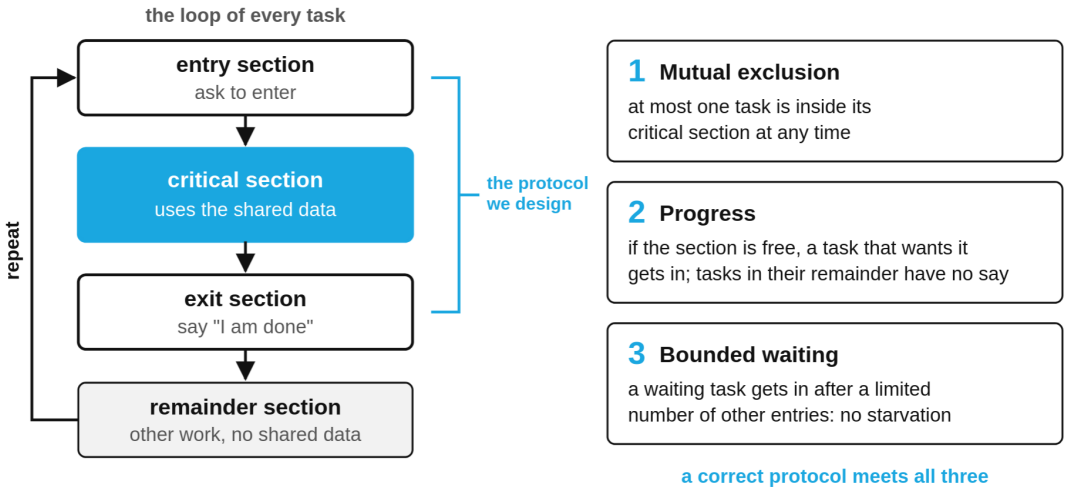
We no have to talk about **synchronization**.

An operation is **atomic** if it runs as one step that nobody can cut in two: no other task runs in the middle of it. Some of our operations have to be atomic, or at least be atomic in regard to some OS resource (say, a peripheral): have **mutual exclusion** on it.

Currently, we have no OS primitive to enforce this. Let's fix this.

- A **shared resource** is anything two or more tasks use: a variable, a buffer, a device such as the UART;
- A **race condition** is a bug where the result depends on the order in which the steps of the tasks happen to run;
- An **interleaving** is one possible order of all the steps of the tasks. With preemption, the scheduler can produce any interleaving;
- A **critical section** is a piece of code that uses a shared resource and must not run in the middle of another.

To remove the race, we do not need to forbid switches: we need to stop a second task from entering its critical section while the first one is inside, in regards to one common shared resource.



We see that whatever protocol we define must meet:

- **Mutual exclusion**: at most one task is inside its critical section at any time. This is the whole point of the protocol;
- **Progress**: if nobody is inside and some tasks want to enter, one of them gets in; a task busy in its remainder section cannot stop them. A protocol that keeps the door always closed is safe but useless;
- **Bounded waiting**: once a task asks to enter, the other tasks can enter only a limited number of times before it does. Otherwise it may starve.

We assume nothing about speed: the protocol must work for any interleaving, any quantum, any number of interrupts.

Let's see a few possible solutions.

1.1.1 Mask interrupts

On one CPU, a task loses the CPU only in two ways: an interrupt (the tick, then PendSV), or its own call (`yield()`, `sleep()`, ...). If the critical section masks interrupts and makes no such call, no other task can run inside it.

This, though, is quite limited and applies to small things we basically alerady saw, *inside* the kernel:

- **Privileged only**: `cpsid i` changes PRIMASK only in privileged mode. Our tasks are unprivileged, and in unprivileged mode the instruction is simply ignored: no fault, no effect. That is on purpose: a task that could mask interrupts could stop the tick and keep the CPU for ever, and the whole protection layer would be useless;
- **Latency**: while interrupts are masked, every interrupt waits, even an urgent one that has nothing to do with the shared data. The section must be very short;
- **One CPU** only: on a chip with two cores, masking on core 0 does nothing to core 1, which keeps running and can enter the same section at the same time.

So masking is a good tool inside the kernel, for a few instructions. Tasks need something else: we can't give them access to interrupt masking as is.

1.1.2 Test-and-set flags

A shared flag in memory is useless as it suffers from the exact problem we're trying to solve. We can do with some hardware help. The fix needs the read and the write of the flag to be one atomic step: an atomic read-modify-write. The classic one is **test-and-set**: in one step, read the old value of a word and write 1 into it.

If test-and-set returns 0, the word was free and you set it: you hold the lock. If it returns 1, someone else holds it, and your write of 1 changed nothing. Two tasks can no longer both see 0: the first one's read and write happen together. Many CPUs have such an instruction (x86 has `xchg`, which swaps a register and a word). The Cortex-M3 has none: it does it in another way.

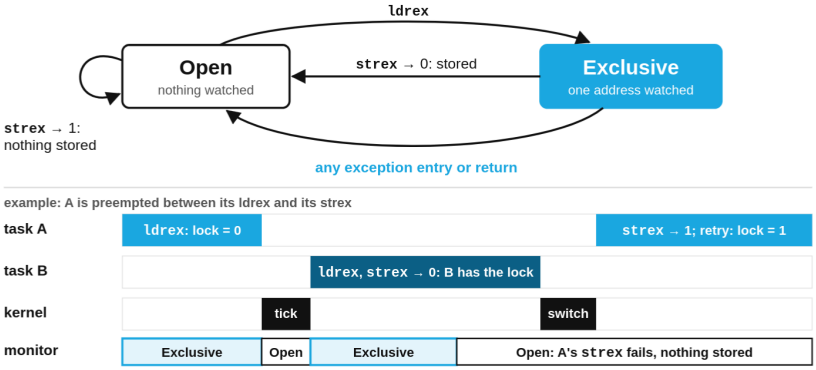
The Cortex-M3 splits the atomic step into two instructions that work together:

```
1 ldrex r1, [r0]      @ r1 = *r0, and start watching: "exclusive access
    begins"
2 strex r2, r1, [r0] @ if nothing happened since ldrex: *r0 = r1, r2 = 0
3                     @ else: store nothing, r2 = 1 (failed)
```

- `ldrex` (*load exclusive*) is a normal load that also says "I want to update this word";
- `strex` (*store exclusive*) stores only if nothing could have touched the word since the ldrex ; its first register tells you if it did (0) or not (1).

If `strex` fails, you simply start again from the `ldrex`- Between the two you can compute anything: test the old value, add 1, ... So the pair gives test-and-set, an atomic increment, and more.

The exclusiveness is ensured by a small piece of hardware: the **exclusive monitor**.



Two states, *Open* and *Exclusive*: `ldrex` sets Exclusive; `strex` stores only in Exclusive, then sets Open. Any exception entry or return sets it to Open (tick, UART interrupt, PendSV, system call), even without a switch. On one CPU another task runs only after an exception: a preempted `strex` fails and A retries.

Another thing to note: to be fast, a CPU and its memory system may let memory accesses complete in a different order than in the program (on bigger cores than ours, very often; the compiler may reorder them too). For a lock that would be fatal: a write of the critical section could become visible after the unlock store, when another task already holds the lock.

`dmb` (*data memory barrier*) allows use to fence this: every memory access before it completes before any memory access after it. The rule is to fence on memory after taking the lock (the section's accesses cannot move before it) and before releasing it. (they cannot move after it). On one Cortex-M3 the barrier costs almost nothing and rarely changes anything, but with it the lock stays correct on every ARM core.

What we implemented is basically a **spinlock**: while the lock is taken, the task stays in its loop and tries again and again. Waiting by looping on the CPU is busy waiting: it can be done by unprivileged tasks, but it's also really wasteful.

If we have the right primitives available, it's better to do **spin-then-yield**. That is, if the lock is taken, yield the task: the holder runs, frees the lock, and you try again at your next turn.

1.1.3 Semaphores

A **semaphore** S is an integer counter, plus a list of tasks blocked on it. It has two operations, both atomic:

- **Wait**: if S's count is above zero, subtract one from it. Otherwise block on the blocked list;
- **Signal**: if the blocked list holds blocked tasks, free the oldest one. Otherwise add one from S's count.

A **mutex** (from *mutual exclusion*) is a lock with an owner: the task that locked it is the only one allowed to unlock it. Two operations:

- **Lock**: if free, the caller becomes the owner; if taken, the caller waits (usually blocked, like a semaphore);
- **Unlock**: allowed only to the owner; the next waiting task becomes the owner. An unlock by another task is an error the kernel can report.

If you think about it hard enough, kernel-side a mutex is just a semaphore initialized at one and where only who moves past the wait is allowed to signal. User-side we need stronger semantics.